

Laporan Tugas Kecil 2

IF2211 Strategi Algoritma



Dipersiapkan oleh:

13524015 - Mahatma Brahmana

13524055 - Junior Natra Situmorang

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

1. Algoritma Divide and Conquer

Pada tugas ini, algoritma Divide and Conquer (D&C) digunakan sebagai strategi utama untuk melakukan pembagian ruang dari sebuah objek 3D menjadi kumpulan voxel (kubus kecil). Secara keseluruhan, kita memecah masalah besar, yaitu menentukan isi dari seluruh ruang bounding box objek, menjadi sub masalah yang lebih kecil secara rekursif hingga mencapai resolusi yang diinginkan.

Langkah pertama, yaitu Divide, direpresentasikan oleh fungsi `GetChildBoundaries`. Di sini, sebuah kubus induk (`Boundary`) dipecah secara matematis menjadi 8 anak yang lebih kecil (sesuai dengan octree). Proses pembagian ini dilakukan dengan menghitung titik pusat (`Center`) baru untuk setiap anak dan membagi dua panjang sisinya (`Half`). Dengan cara ini, kita secara sistematis membagi ruang koordinat 3D menjadi area-area yang lebih spesifik untuk diproses lebih lanjut.

Langkah kedua adalah Conquer, yang dijalankan secara rekursif di dalam fungsi `Build`. Pada tahap ini, setiap node yang sudah dibagi tadi akan diperiksa apakah mengandung bagian dari objek atau tidak melalui fungsi `IsIntersecting`. Jika suatu node ternyata kosong (tidak bersinggungan dengan face manapun), kita melakukan penghentian rekursi lebih awal (`NodesSkipped`), karena tidak ada hasil yang bisa didapat dari pemecahan node tersebut. Namun, jika node tersebut berisi bagian objek (face), kita akan terus memecah ruang tersebut sampai mencapai leaf node, yaitu ketika kedalaman rekursi sudah menyentuh `MaxDepth`.

Terakhir, pada tahap Combine, alih-alih menggabungkan struktur data secara manual di akhir, proses penggabungan terjadi melalui penulisan langsung ke file `.obj` oleh fungsi `writeBoxToFile` setiap kali leaf node tercapai. Kumpulan boundary (kubus) kecil yang berhasil ditulis inilah yang menyusun kembali bentuk visual dari hasil perubahan yang sudah dilakukan.

Berikut adalah pseudocode dari fungsi yang menggunakan algoritma Divide and Conquer

```
FUNCTION GetChildBoundaries(b: Boundary) → array of Boundary
newHalf  $\leftarrow$  b.Half / 2

children[0]  $\leftarrow$  (center: (x - newHalf, y - newHalf, z - newHalf))
children[1]  $\leftarrow$  (center: (x + newHalf, y - newHalf, z - newHalf))
children[2]  $\leftarrow$  (center: (x - newHalf, y + newHalf, z - newHalf))
children[3]  $\leftarrow$  (center: (x + newHalf, y + newHalf, z - newHalf))
children[4]  $\leftarrow$  (center: (x - newHalf, y - newHalf, z + newHalf))
children[5]  $\leftarrow$  (center: (x + newHalf, y - newHalf, z + newHalf))
children[6]  $\leftarrow$  (center: (x - newHalf, y + newHalf, z + newHalf))
children[7]  $\leftarrow$  (center: (x + newHalf, y + newHalf, z + newHalf))

RETURN children
END FUNCTION
```

```
FUNCTION Build(currentBox, faces, vertices, depth, filePath, file,
offset) → OctreeNode
  IF depth = 0 THEN
    file  $\leftarrow$  CREATE filePath
    offset  $\leftarrow$  1
```

```

END IF

IF NOT IsIntersecting(currentBox, faces, vertices) THEN
    NodesSkipped[depth] ← NodesSkipped[depth] + 1
    RETURN NULL
END IF

IF depth = MaxDepth THEN
    TotalLeaf ← TotalLeaf + 1
    NodesCount[depth] ← NodesCount[depth] + 1

    writeBoxToFile(file, currentBox, offset)

    RETURN Node(currentBox, IsLeaf = TRUE, Depth = depth)
END IF

node ← NEW OctreeNode
node.Box ← currentBox
node.Depth ← depth

NodesCount[depth] ← NodesCount[depth] + 1

children ← GetChildBoundaries(currentBox)

FOR i ← 0 TO 7 DO
    node.Children[i] ← Build(children[i], faces, vertices,
depth + 1, filePath, file, offset)
END FOR

RETURN node
END FUNCTION

```

```

FUNCTION IsIntersecting(box, faces, vertices) → boolean
    FOR each face f in faces DO
        IF f kosong THEN
            CONTINUE
        END IF

        hitung bounding box face:
            fMinX, fMaxX
            fMinY, fMaxY
            fMinZ, fMaxZ

        hitung bounding box box:
            boxMinX, boxMaxX
            boxMinY, boxMaxY
            boxMinZ, boxMaxZ

        IF overlap antara face dan box THEN
            RETURN TRUE
        END IF
    END FOR

    RETURN FALSE
END FUNCTION

```

```

FUNCTION writeBoxToFile(file, box, offset)
hitung 8 titik sudut kubus

FOR setiap titik v DO
TULIS "v x y z" ke file
END FOR

buat 12 segitiga (faces)

FOR setiap face f DO
TULIS "f i j k" ke file
END FOR

offset ← offset + 8
END FUNCTION

```

Build adalah fungsi utama yang digunakan untuk penggunaan algoritma Divide and Conquer ini. Dapat dilihat dari fungsi build diatas, fungsi isIntersecting (Conquer) pertama kali dilakukan. Lalu dilanjutkan dengan, writeBoxToFile (Combine) dan setelah itu GetChildBoundaries (Divide).

Alasan Conquer dulu baru Combine baru Divide adalah supaya kita tidak kerja dua kali. Kita mulai dengan Conquer (cek tabrakan) untuk memastikan apakah kotak tersebut ada isinya atau kosong; kalau kosong, kita bisa langsung berhenti. Setelah itu, kita masuk ke Combine (tuliskan hasil), di mana kalau kotak tersebut ada isinya dan sudah mencapai ukuran terkecil yang kita mau, kita langsung simpan sebagai bagian dari model 3D. Terakhir, baru kita lakukan Divide (bagi delapan), yaitu memecah kotak tersebut menjadi 8 bagian kecil hanya jika kotak itu memang ada isinya tapi ukurannya masih terlalu besar. Dengan urutan ini, komputer jadi sangat efisien karena dia hanya akan membagi kotak di tempat yang benar benar ada objeknya saja, bukannya asal membagi ruang kosong yang tidak perlu.

2. Test Case Input / Output

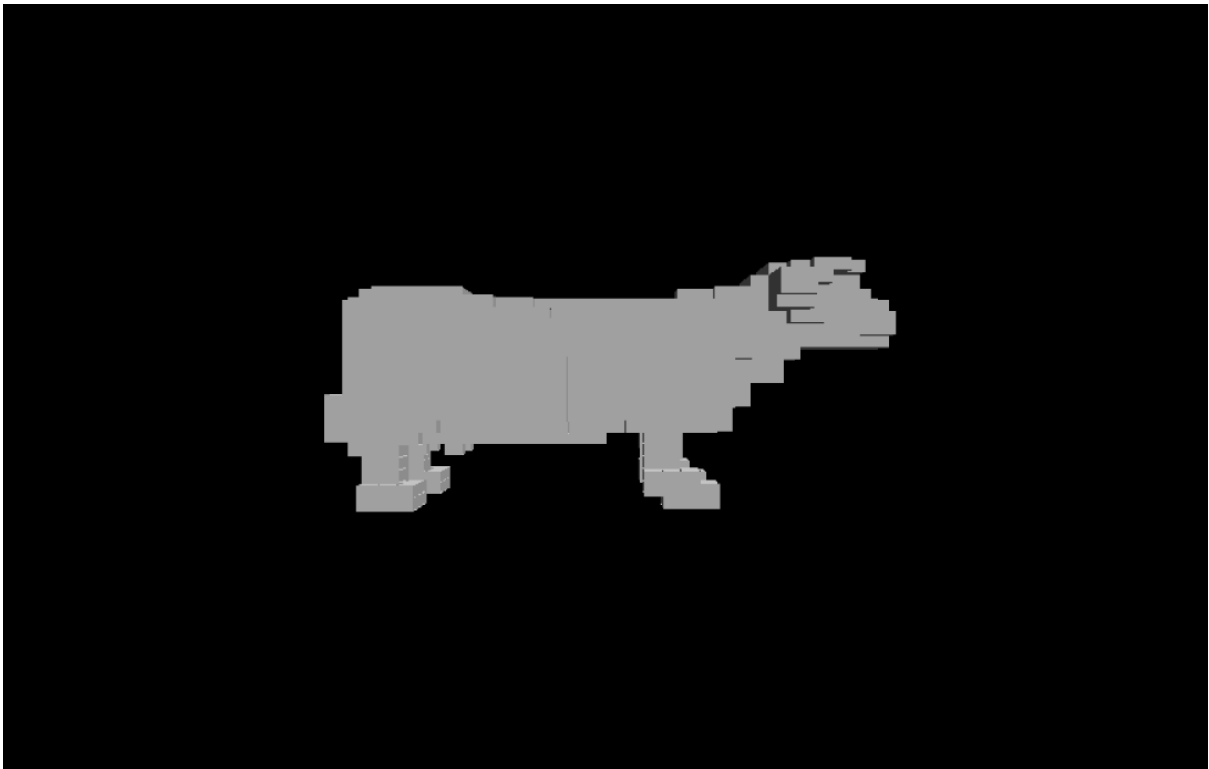
a. cow.obj & Max_Depth = 5

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go cow.obj 5
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 4583 vertices and 5804 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 1764
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 20 nodes, Skipped 44 nodes
Depth 3: Created 96 nodes, Skipped 64 nodes
Depth 4: Created 372 nodes, Skipped 396 nodes
Depth 5: Created 1764 nodes, Skipped 1212 nodes
Octree construction took 0.5971089 seconds
Octree Max Depth: 5
Do you want to view the output? (Y/n) Y
```



b. cow.obj & Max_Depth = 7

```

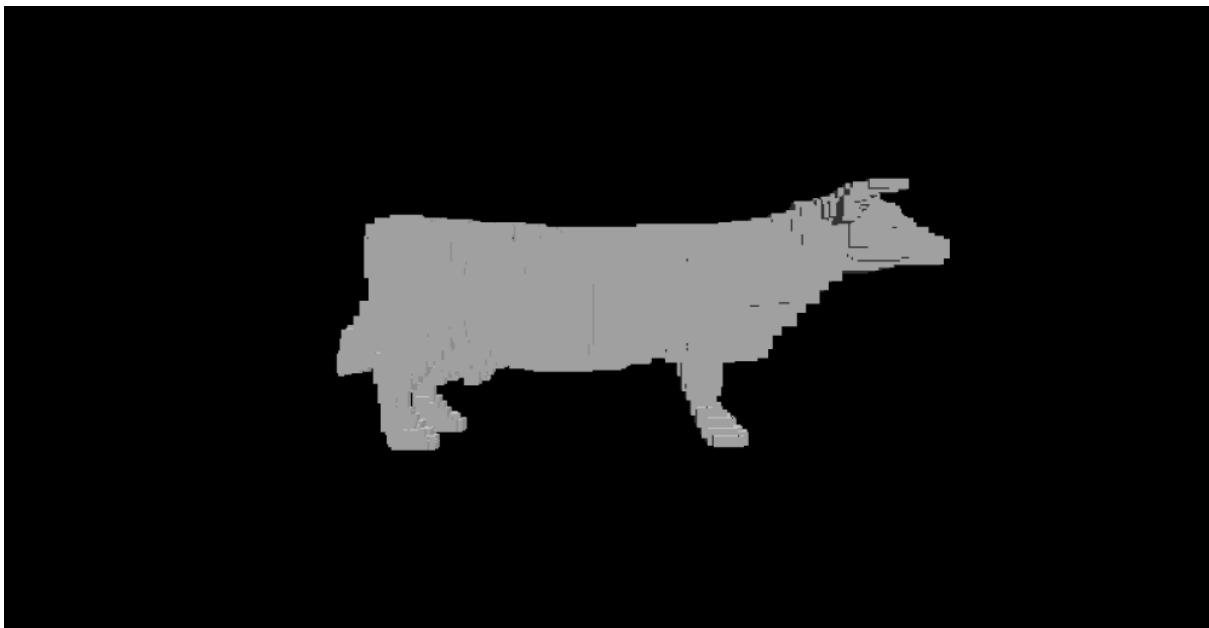
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go cow.obj 7
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 4583 vertices and 5804 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 49071
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 20 nodes, Skipped 44 nodes
Depth 3: Created 96 nodes, Skipped 64 nodes
Depth 4: Created 372 nodes, Skipped 396 nodes
Depth 5: Created 1764 nodes, Skipped 1212 nodes
Depth 6: Created 8797 nodes, Skipped 5315 nodes
Depth 7: Created 49071 nodes, Skipped 21305 nodes
Octree construction took 12.7646364 seconds
Octree Max Depth: 7
Do you want to view the output? (Y/n) Y

```



c. line.obj & Max_Depth = 5

```

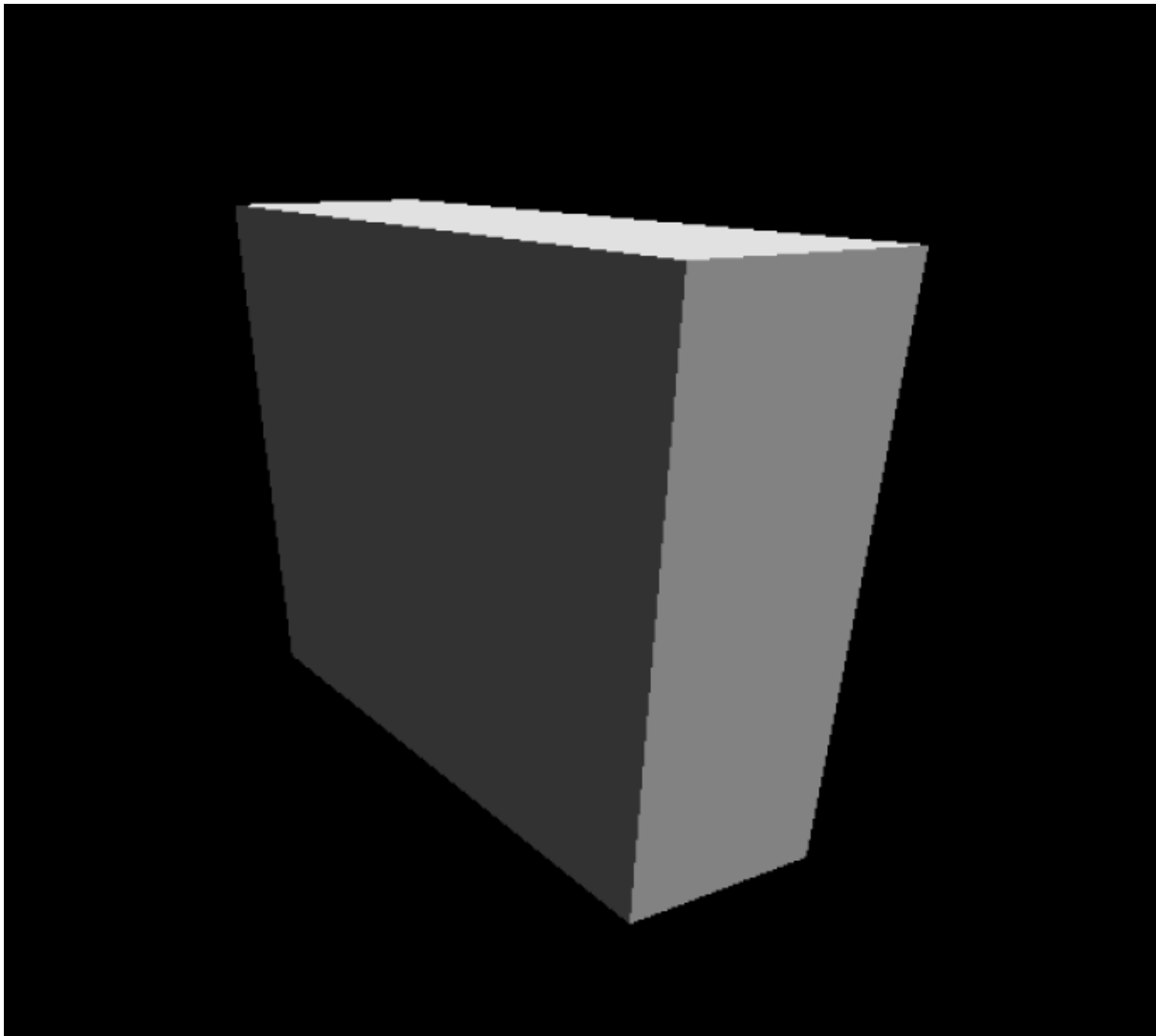
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go line.obj 5
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 3 vertices and 1 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 7168
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 32 nodes, Skipped 32 nodes
Depth 3: Created 128 nodes, Skipped 128 nodes
Depth 4: Created 896 nodes, Skipped 128 nodes
Depth 5: Created 7168 nodes, Skipped 0 nodes
Octree construction took 0.4041689 seconds
Octree Max Depth: 5
Do you want to view the output? (Y/n) Y

```



d. line.obj & Max_Depth = 7

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go line.obj 7
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 3 vertices and 1 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 359424
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 32 nodes, Skipped 32 nodes
Depth 3: Created 128 nodes, Skipped 128 nodes
Depth 4: Created 896 nodes, Skipped 128 nodes
Depth 5: Created 7168 nodes, Skipped 0 nodes
Depth 6: Created 48384 nodes, Skipped 8960 nodes
Depth 7: Created 359424 nodes, Skipped 27648 nodes
Octree construction took 19.7293144 seconds
Octree Max Depth: 7
```



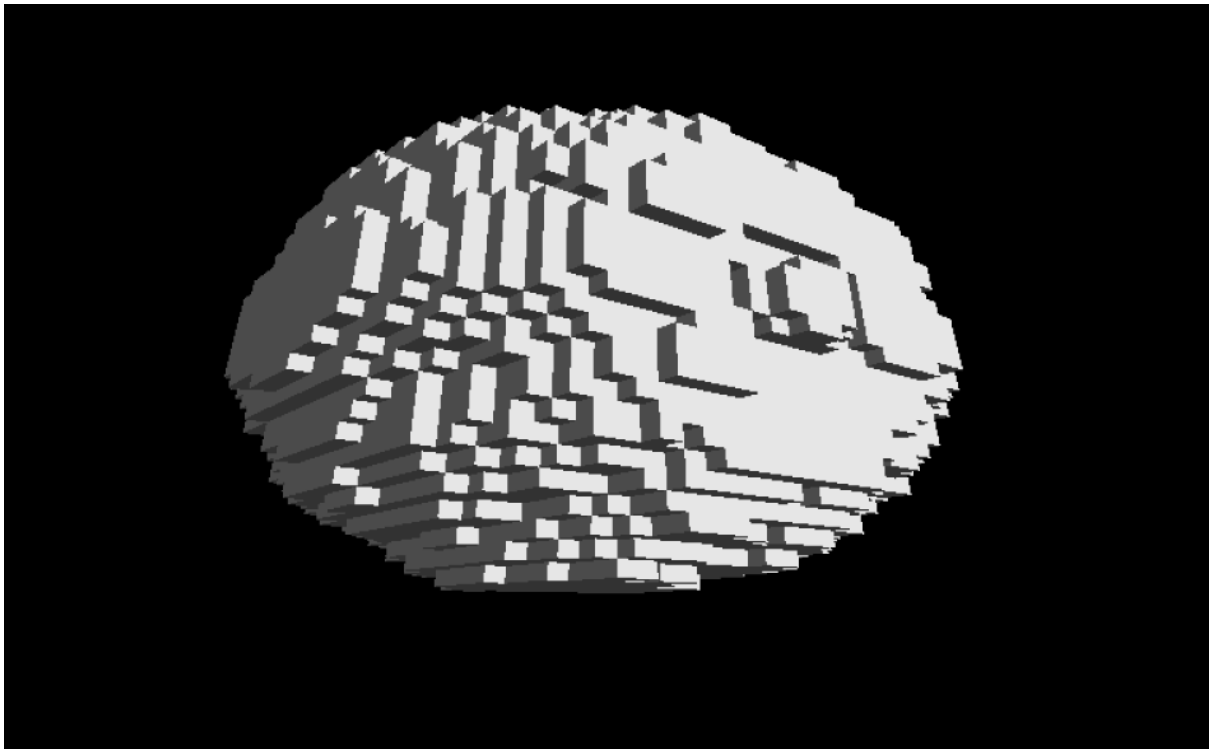
e. pumpkin.obj & Max_Depth = 5

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go pumpkin.obj 5
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 5002 vertices and 10000 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 6421
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 56 nodes, Skipped 8 nodes
Depth 3: Created 300 nodes, Skipped 148 nodes
Depth 4: Created 1345 nodes, Skipped 1055 nodes
Depth 5: Created 6421 nodes, Skipped 4339 nodes
Octree construction took 3.5332608 seconds
Octree Max Depth: 5
Do you want to view the output? (Y/n) Y
```



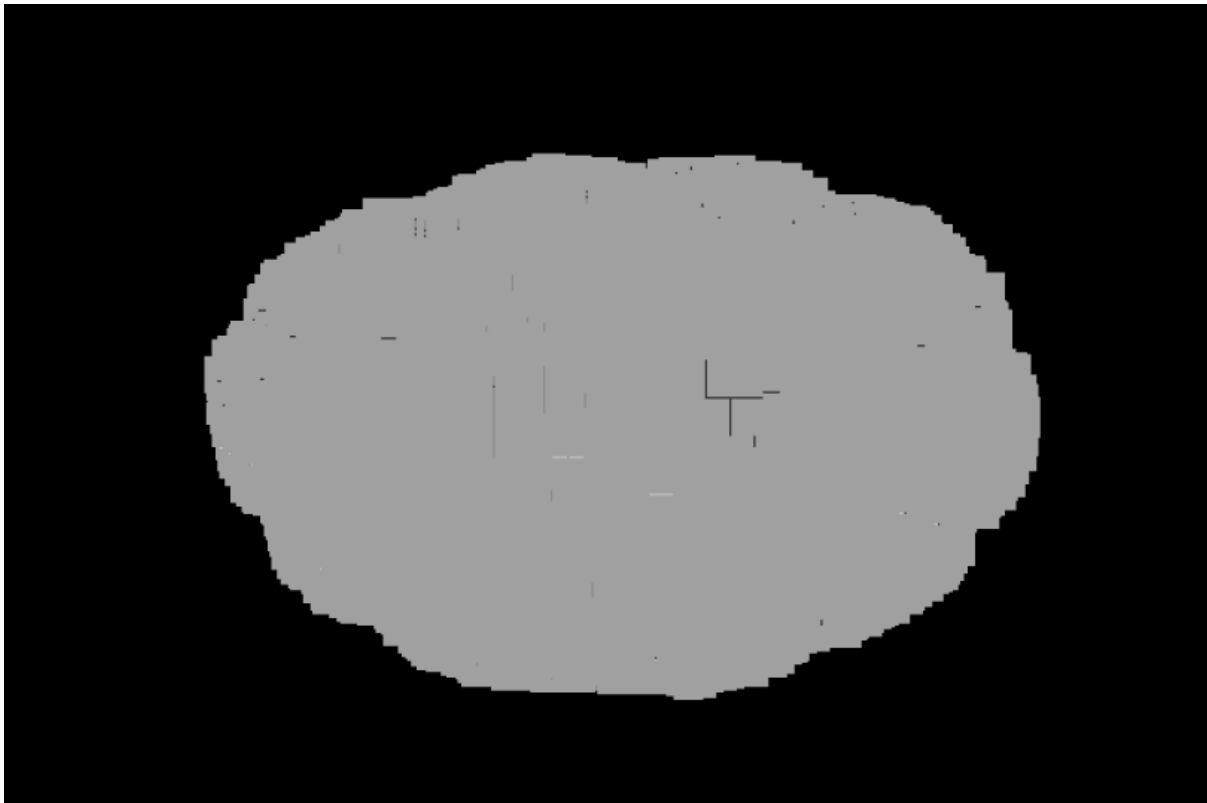
f. pumpkin.obj & Max_Depth = 7

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go pumpkin.obj 7
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 5002 vertices and 10000 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 190455
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 56 nodes, Skipped 8 nodes
Depth 3: Created 300 nodes, Skipped 148 nodes
Depth 4: Created 1345 nodes, Skipped 1055 nodes
Depth 5: Created 6421 nodes, Skipped 4339 nodes
Depth 6: Created 33134 nodes, Skipped 18234 nodes
Depth 7: Created 190455 nodes, Skipped 74617 nodes
Octree construction took 81.7170674 seconds
Octree Max Depth: 7
Do you want to view the output? (Y/n) Y
```



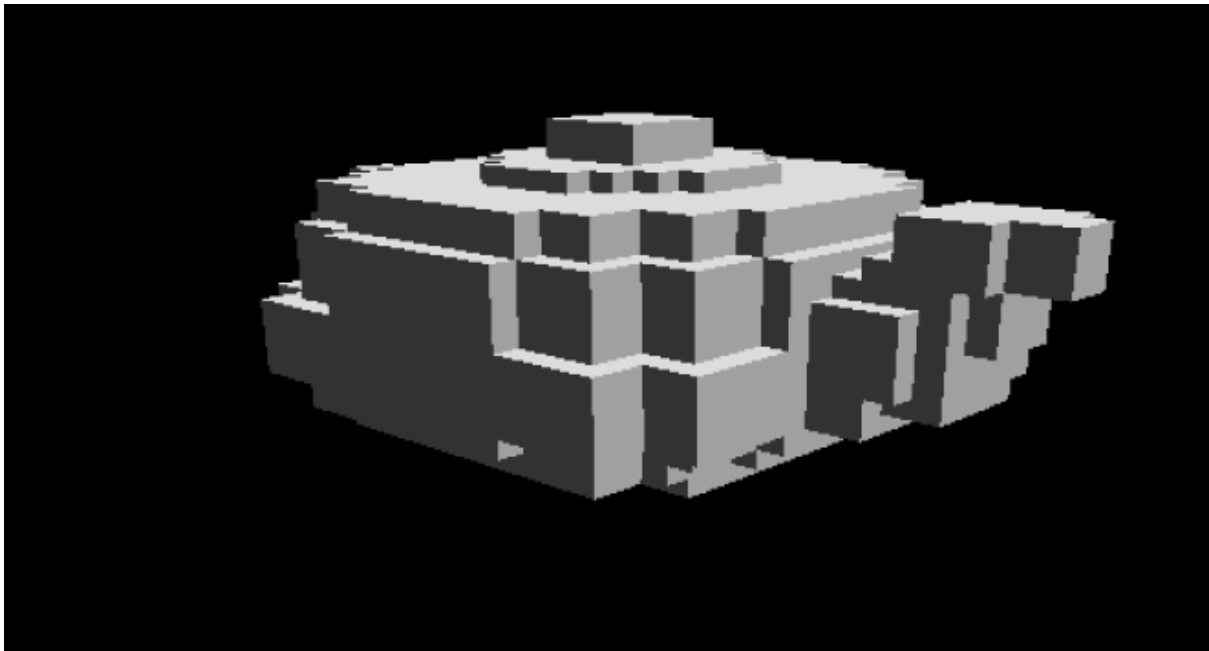
g. teapot.obj & Max_Depth = 5

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go teapot.obj 5
Warning: Unrecognized line format at line 1051: vn -1.37744e-09 0.984907 -0.173084
Warning: Unrecognized line format at line 1052: vn -2.79781e-10 0.925108 -0.379703
Warning: Unrecognized line format at line 1053: vn -2.55497e-09 0.850068 -0.526673
Warning: Unrecognized line format at line 1054: vn 0.0902162 0.971775 -0.217978
Warning: Unrecognized line format at line 1055: vn 0.0659615 0.984977 -0.15959
Warning: Unrecognized line format at line 1056: vn 0.14474 0.925312 -0.350498
Warning: Unrecognized line format at line 1057: vn 0.201474 0.850653 -0.48559
Warning: Unrecognized line format at line 1058: vn 0.166631 0.971838 -0.166631
Warning: Unrecognized line format at line 1059: vn 0.121908 0.985026 -0.121908
Warning: Unrecognized line format at line 1060: vn 0.267668 0.925585 -0.267668
Warning: Unrecognized line format at line 1061: vn 0.371315 0.851029 -0.371315
Warning: Unrecognized line format at line 1062: vn 0.217978 0.971775 -0.0902162
Warning: Unrecognized line format at line 1063: vn 0.15959 0.984977 -0.0659615
Warning: Unrecognized line format at line 1064: vn 0.350498 0.925312 -0.14474
Warning: Unrecognized line format at line 1065: vn 0.48559 0.850653 -0.201474
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 530 vertices and 1024 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 3044
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 26 nodes, Skipped 38 nodes
Depth 3: Created 128 nodes, Skipped 80 nodes
Depth 4: Created 573 nodes, Skipped 451 nodes
Depth 5: Created 3044 nodes, Skipped 1540 nodes
Octree construction took 0.2139515 seconds
Octree Max Depth: 5
Do you want to view the output? (Y/n) Y
```



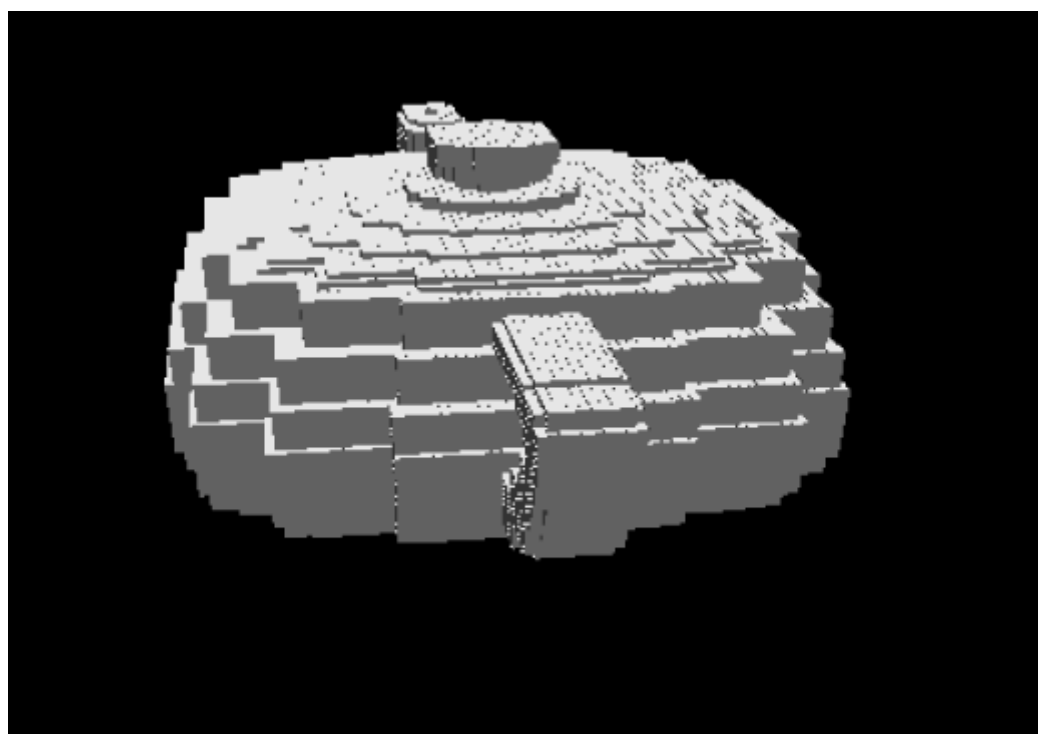
h. teapot.obj & Max_Depth = 7

```
PS D:\KULIAH\SEMESTER 4\STIMA\TUCIL\TUCIL2\Tucil2_13524015_13524055\src> go run main.go octree.go viewer.go teapot.obj 7
Warning: Unrecognized line format at line 1053: vn -2.55497e-09 0.850068 -0.526673
Warning: Unrecognized line format at line 1054: vn 0.0902162 0.971775 -0.217978
Warning: Unrecognized line format at line 1055: vn 0.0659615 0.984977 -0.15959
Warning: Unrecognized line format at line 1056: vn 0.14474 0.925312 -0.350498
Warning: Unrecognized line format at line 1057: vn 0.201474 0.850653 -0.48559
Warning: Unrecognized line format at line 1058: vn 0.166631 0.971838 -0.166631
Warning: Unrecognized line format at line 1059: vn 0.121908 0.985026 -0.121908
Warning: Unrecognized line format at line 1060: vn 0.267668 0.925585 -0.267668
Warning: Unrecognized line format at line 1061: vn 0.371315 0.851029 -0.371315
Warning: Unrecognized line format at line 1062: vn 0.217978 0.971775 -0.0902162
Warning: Unrecognized line format at line 1063: vn 0.15959 0.984977 -0.0659615
Warning: Unrecognized line format at line 1064: vn 0.350498 0.925312 -0.14474
Warning: Unrecognized line format at line 1065: vn 0.48559 0.850653 -0.201474
Enter output .obj path (example: result.obj or ../output/result.obj): res

Parsing Started
Parsed 530 vertices and 1024 faces from the OBJ file.

Process Started
Process Done
Voxels exported to ../output/res.obj successfully!

Octree Construction Result:
Total Voxels created: 114489
Depth 0: Created 1 nodes, Skipped 0 nodes
Depth 1: Created 8 nodes, Skipped 0 nodes
Depth 2: Created 26 nodes, Skipped 38 nodes
Depth 3: Created 128 nodes, Skipped 80 nodes
Depth 4: Created 573 nodes, Skipped 451 nodes
Depth 5: Created 3044 nodes, Skipped 1540 nodes
Depth 6: Created 17892 nodes, Skipped 6460 nodes
Depth 7: Created 114489 nodes, Skipped 28647 nodes
Octree construction took 7.2614946 seconds
Octree Max Depth: 7
Do you want to view the output? (Y/n) Y
```



3. Lampiran

Seluruh source code dan berkas pendukung dalam pengerjaan tugas ini tersedia pada repositori GitHub yang dapat diakses melalui tautan berikut:

https://github.com/Jerannn24/Tucil2_13524015_13524055

Pernyataan tidak melakukan kecurangan yang ditandatangani dengan format sebagai berikut:

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Mahatma Brahmana - 13524015



Junior Natra Situmorang - 13524055

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan <i>concurrency</i>		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj	✓	

4. Reference

- [1] E. Nevala, "Introduction to Octrees," *Wobblyduckstudios.com*, 2018. Available: <https://wobblyduckstudios.com/Octrees.php>. [Accessed: Mar. 23, 2026]
- [2] R. Munir, "Strategi Algoritma," *Itb.ac.id*, 2025. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/stima25-26.htm>